STM32H750 电赛智能车 – 原理与实现

3.3 视觉处理入门:
基础视觉功能的代码演示

InnoLegend 技术团队

目 录

CONTENTS

1 sensor传感器模块

2 time时钟模块的使用

3 image对象

4 色块检测

第一部分

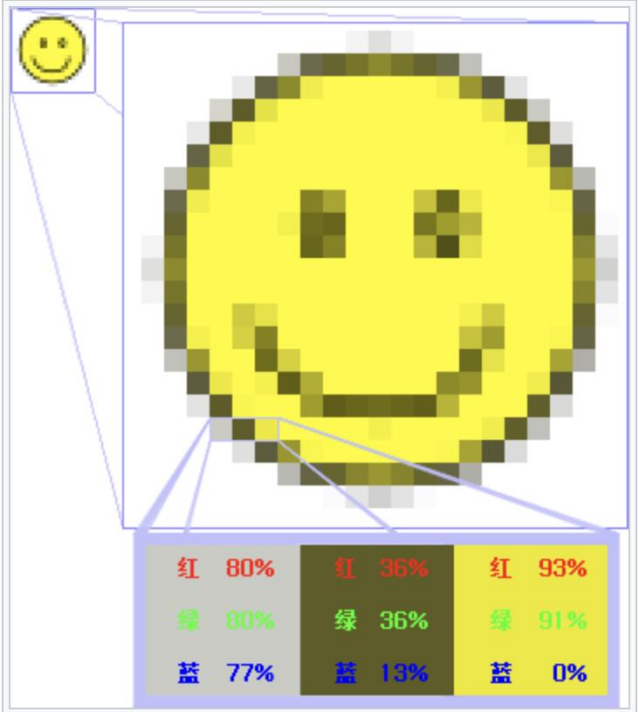
sensor传感器模块

Sensor module

1.1 图像处理背景知识介绍

InnoLegend

- 图像(image)：就是用很多小点（像素）组成的一张照片或画面。
- 像素(pixel)：图像中的最小点，就像马赛克里的单个砖块，每个点都有颜色。
- 分辨率：表示图像有多少个像素，比如320×240就有320列240行这么多点，分辨率越高，图像越清晰。
- 帧率：摄像头每秒钟拍摄多少张图画，比如30帧每秒（30fps），数字越大，画面越流畅。
- 色彩：颜色是通过红(Red)、绿(Green)、蓝(Blue)三种光的强度混合形成的，摄像头会记录每个像素点这三种颜色的明暗程度。



RGB色彩表示的笑脸

1.2 导入sensor与使用

InnoLegend

sensor模块：初始化和控制摄像头，采集并提供图像数据给程序处理

```
importsensor,image
# 初始化摄像头
sensor.reset()
sensor.set_pixformat(sensor.RGB565)# 使用彩色模式
sensor.set_framesize(sensor.QVGA)#320*240
sensor.skip_frames(time=2000)#缓冲2s，等待感光元件稳定
sensor.set_auto_gain(False)#关闭自动增益
sensor.set_auto_whitebal(False)#关闭自动白平衡
sensor.set_hmirror(True)#水平翻转
sensor.set_vflip(True)#垂直翻转
while(True):
img = sensor.snapshot()
```

如何设置sensor模块? → 取决于我们需要怎么样的图像

1.3 快照：sensor.snapshot()

InnoLegend

sensor.snapshot()就像是用摄像头"按下快门"拍照一样！

当你运行这行代码时，OpenMV会：

- 立即从摄像头捕获当前画面
- 将这张"照片"保存在内存中
- 返回一个叫做`image`的特殊对象，这个对象包含了刚拍摄的图像

小贴士：

1. sensor.snapshot()是几乎所有OpenMV视觉程序的核心，因为它提供了待处理的图像。
2. 每次调用 sensor.snapshot()都会获取一个全新的画面，之前的画面会被"丢弃"。
3. 在循环中频繁调用 sensor.snapshot()可以实现类似视频处理的效果。
4. 保存照片，可以使用：img.save

第二部分

time时钟模块的使用

Use of time clock module

2.1 time时钟模块

time模块主要用于控制程序的时间管理，计算帧率

```
importsensor,image,time
# 初始化摄像头
sensor.reset()
sensor.set_pixformat(sensor.RGB565)
sensor.set_framesize(sensor.QVGA)
sensor.skip_frames(time=2000)
clock = time.clock()#创建时钟对象以跟踪fps
while(True):
clock.tick()#更新时钟
img = sensor.snapshot()
clock.sleep(time=1000)#暂停1s
print(clock.fps())# 打印帧率
```

注意：在IDE中图像缓冲区显示的帧率会和实际脱机运行时有差异，IDE中的帧率通常更低!

第三部分

image对象

Image Object

3.1 image介绍

InnoLegend

image对象 ⇨ 一个带有特殊能力的照片

这个image对象有很多"超能力"（也就是函数），拍完照片后，你可以对这个image对象做各种操作

- ✓ 在图像上画线、画圆、画矩形
- ✓ 查找特定颜色的区域
- ✓ 检测线条、形状
- ✓ 识别物体
- ✓ 等等...

3.2 像素操作与图像处理

InnoLegend

像素操作：

- pixel_value = img.get_pixel(10, 10) #获取像素点(10, 10) 的像素值
- img.set_pixcel(10, 10, (255, 0, 0)) #设置像素点(10, 10) 的像素值为红色

图像处理：

- img.invert() #反转图像

- img.nand(image) # 与另一个图片进行与非（NAND）运算
- img.nor(image) # 与另一个图片进行或非（NOR）运算
- img.xor(image) # 与另一个图片进行异或（XOR）运算
- img.xnor(image) # 与另一个图片进行异或非（XNOR）运算

- img.difference(image)#计算差异图

3.3 图像绘制

InnoLegend

画线：

- img.draw_line((20, 30, 40, 50))

(x1, y1, x2, y2)，表示线的起点和终点坐标

- img.draw_line((80, 50, 100, 100), color=(255, 0, 0))

画矩形：

- img.draw_rectangle((20, 30, 41, 51), color=(255, 0, 0))

(x, y, w, h)，x、y为矩形左上角坐标，w和h分别为矩形的宽度和高度

画圆：

- img.draw_circle(50, 50, 30)

（x，y，r）圆心与半径

画十字：

- img.draw_cross(90, 60, size=10)

（x，y，size）x、y为十字中心坐标，size控制十字的大小

显示文字：

- img.draw_string(10, 10, "hello world!")

（x，y，string）x、y为文字框左上角的坐标，string为要显示的文字

3.4 高级图像分析

InnoLegend

线性回归

```
line=img.get_regression
([(0,100)],robust=True)
```

这个函数用于寻找图像中的线条。如果找到线条，返回的line对象包含线条的各种信息，如角度、位置等。

色块检测

```
for blob in
img.find_blobs([red_threshold],
pixels_threshold=200,
area_threshold=200,merge=True):
```

函数返回一个blob对象的列表，每个blob对象包含了色块的位置、大小、形状等信息。

第四部分

色块检测

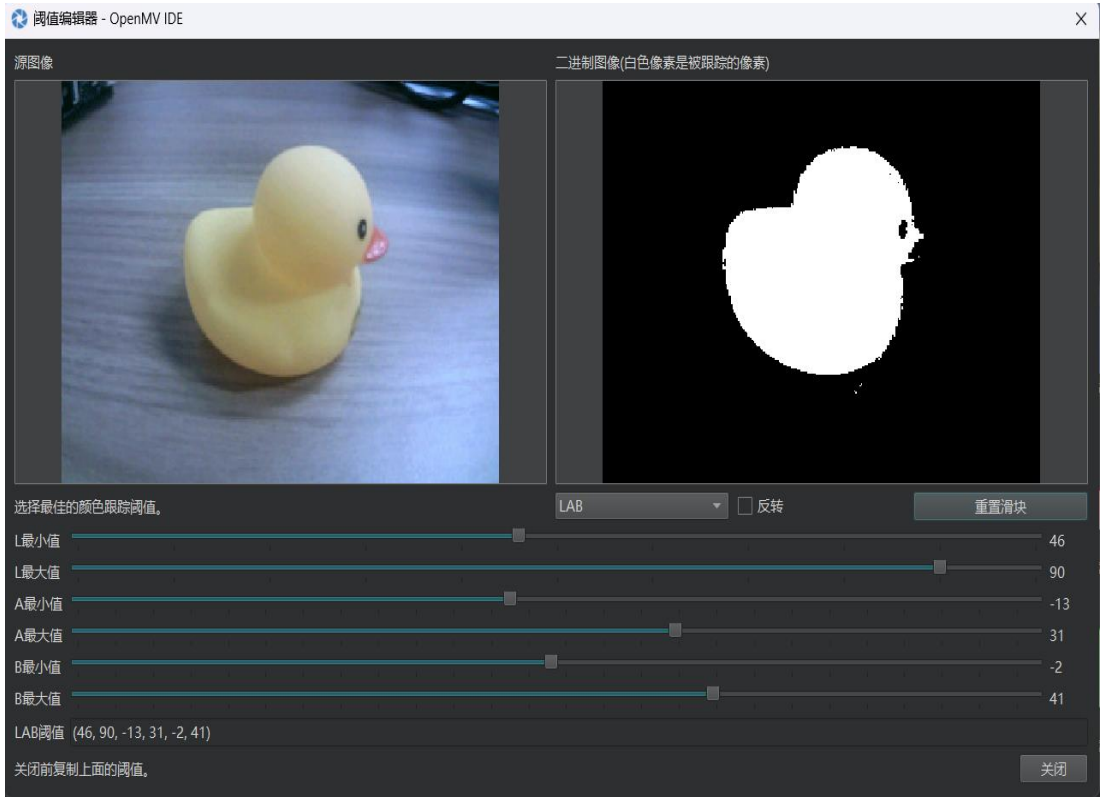
Color block detection

4.1 设置颜色阈值

InnoLegend

使用OpenMV IDE中的阈值编辑器：

1. 运行一个简单程序让摄像头显示图像
2. 打开"工具→机器视觉→阈值编辑器"
3. 点击"Frame Buffer"获取当前图像
4. 调整六个滑块，直到你要检测的颜色变成白色，其他颜色变成黑色
5. 复制显示的阈值值



阈值编辑器示例

4.2 find_blobs函数详解

InnoLegend

```
for blob in
img.find_blobs([red_threshold],pixels_threshold=200,
area_threshold=200,merge=True):
```

- 颜色阈值参数：red_threshold = (30, 100, 15, 127, 15, 127)
- 像素阈值参数：pixels_threshold=200
- 面积阈值参数：area_threshold=200
- 合并参数：merge=True

4.3 blob对象的常用方法

InnoLegend

每个找到的色块都是一个blob对象，它包含了很多有用的信息：

- blob.rect() - 返回色块的矩形边界 (x, y, w, h)
- blob.cx() - 返回色块中心的x坐标
- blob.cy() - 返回色块中心的y坐标
- blob.area() - 返回色块的面积
- blob.pixels() - 返回色块包含的像素数量
- blob.code() - 返回色块的颜色代码（当检测多种颜色时很有用）
- blob.count() - 如果merge=True，返回合并的色块数量
- blob.density() - 返回色块的密度（像素数/面积）

4.4 提高色块检测效果的小技巧

InnoLegend

- 关闭自动增益和白平衡：自动增益和白平衡会导致同一物体在不同光线下颜色变化。
- 精调阈值：使用阈值编辑器反复调整，直到只有目标颜色被识别。
- 使用ROI：如果只需要检测图像中特定区域的色块，可以设置ROI（Region Of Interest）：

```
roi = (0, 0, 160, 120)  # 左上角160×120的区域
blobs = img.find_blobs([red_threshold], roi=roi)
```

- 合理设置阈值参数：根据实际应用调整pixels_threshold和area_threshold，过滤掉干扰。

STM32H750 电赛智能车 – 原理与实现

欢迎各位同学交流、指正

InnoLegend 技术团队